

Estudo Dirigido 1.2

1º Bimestre

Descrição

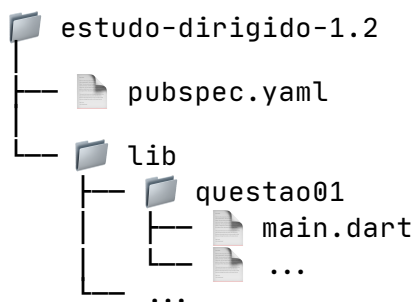
Nesta lista, você vai praticar a criação de classes em Dart com foco em construtores, instanciação de objetos, referências e comportamento da memória durante a execução do programa. Os exercícios foram organizados para que você implemente soluções e observe, na prática, como os objetos são criados, compartilhados, modificados e liberados pelo garbage collector.

Instruções de entrega

Para a submissão do trabalho, siga os passos abaixo:

- Criar um novo repositório no github chamado **programacao-oo**
 - Selecionar para criar o REAME.md
 - Selecionar para criar .gitgnore usando template Dart
- Incluir o **nome dos integrantes** da equipe no README.md (até 3 alunos)
- Compartilhar o do repositório na **planilha** disponibilizada pelo professor (TEAMS)

Para garantir organização, o trabalho deve seguir a estrutura de diretórios abaixo:



Critérios de avaliação

A avaliação será baseada nos seguintes aspectos:

- Uso correto de classes e atributos
- Implementação de construtores
- Implementação de métodos
- Organização e legibilidade do código
- Capacidade de explicar o funcionamento do código

Questões

Questão 1

Lâmpada com construtor sem parâmetros

Crie uma classe Lâmpada com os atributos privados `_ligada` e `_intensidade`. Implemente um construtor sem parâmetros que inicialize a lâmpada desligada e com intensidade 50. Crie métodos `ligar()`, `desligar()`, `aumentar()`, `diminuir()` e `exibirEstado()`.

Questão 2

Produto com construtor posicional

Crie uma classe Produto com `_nome`, `_preco` e `_estoque`. Implemente um construtor com parâmetros posicionais. Adicione os métodos `repor(int qtd)`, `vender(int qtd)` e `exibirFicha()`.

Questão 3

Aluno com parâmetros nomeados obrigatórios

Crie uma classe Aluno com `_ra`, `_nome` e `_notaFinal`. Use construtor com parâmetros nomeados obrigatórios. Implemente `exibirBoletim()` e `aprovado()`.

Referência: <https://dart.dev/language/functions#named-parameters>

Questão 4

Usuário com parâmetros opcionais

Crie uma classe Usuario com `_nome`, `_email` e `_apelido`. O apelido deve ser opcional. Se não for informado, use o próprio nome como apelido. Implemente `exibirPerfil()`.

Referência: <https://dart.dev/language/functions#optional-positional-parameters>

Questão 5

Conta com construtor nomeado

Crie uma classe Conta com `_numero`, `_titular` e `_saldo`. Implemente:

- `Conta.bancaria(String numero, String titular)`
- `Conta.vip(String numero, String titular, double saldoInicial)`

A primeira cria conta com saldo zero. A segunda inicia com saldo informado. Adicione `depositar()`, `sacar()` e `exibirResumo()`.

Referência: <https://dart.dev/language/constructors#named-constructors>

Questão 6

Retângulo com múltiplas formas de construção

Crie uma classe Retangulo com `_largura` e `_altura`. Implemente:

- um construtor padrão com parâmetros,
- um construtor sem parâmetros que crie um quadrado 1x1,
- um construtor nomeado `Retangulo.quadrado(double lado)`.

Inclua `calcularArea()`, `calcularPerimetro()` e `exibirDados()`.

Questão 7

Referência para o mesmo objeto

Crie uma classe Personagem com os atributos privados `_nome`, `_vida` e `_energia`. No main, instancie um objeto e atribua essa mesma instância a uma segunda variável. Em seguida, altere o valor de `_vida` usando a segunda variável e exiba o estado do objeto antes e depois da alteração, com impressões na tela.

```
void main() {  
    Personagem p1 = Personagem('Arthas', 100, 80);  
    Personagem p2 = p1;  
  
    print('Antes da alteração:');  
    p1.exibir();  
    p2.exibir();  
  
    p2.alterarVida(50);  
  
    print('\nDepois da alteração:');  
    p1.exibir();  
    p2.exibir();  
}
```

Após executar o programa, responda:

- a) Por que, ao alterar a segunda variável, a primeira também apresentou a alteração?
- b) Esse comportamento está correto? Explique com base no conceito de referência de objetos em Dart.

Questão 8

Controle remoto e televisão

Crie uma classe Televisao com os atributos privados `_canal` e `_volume`. Depois, crie uma classe ControleRemoto que receba um objeto Televisao no construtor e implemente os métodos `aumentarVolume()`, `diminuirVolume()`, `proximoCanal()` e `canalAnterior()`. No main, crie uma televisão e um controle remoto associado a ela. Use o controle para alterar o estado da televisão e mostre os resultados com impressões na tela.

```
void main() {  
    Televisao tv = Televisao(5, 20);  
    ControleRemoto controle = ControleRemoto(tv);  
  
    print('Estado inicial:');  
    tv.exibir();  
  
    controle.aumentarVolume();  
    controle.proximoCanal();  
  
    print('\nDepois de usar o controle:');  
    tv.exibir();  
}
```

Após executar o programa, responda:

- a) Por que o controle remoto consegue alterar diretamente o estado da televisão?
- b) O controle remoto possui uma cópia da televisão ou uma referência para o mesmo objeto? Como isso pode ser percebido no resultado?

Questão 9

Oficina e carro

Crie uma classe Carro com os atributos privados `_modelo` e `_quilometragem`. Em seguida, crie uma classe Oficina com o método `revisar(Carro carro)`, que altere a quilometragem do carro após um teste de rodagem. No main, crie um carro, envie-o para revisão e exiba seu estado antes e depois da chamada do método.

```
void main() {  
    Carro carro = Carro('Fusca', 12000);  
    Oficina oficina = Oficina();  
  
    print('Antes da revisão:');  
    carro.exibir();  
  
    oficina.revisar(carro);  
  
    print('\nDepois da revisão:');  
    carro.exibir();  
}
```

Após executar o programa, responda:

- a) Por que a quilometragem do carro foi alterada mesmo sem retornar um novo objeto?
- b) O parâmetro recebido pelo método `revisar()` representa um novo carro ou o mesmo objeto criado no main?

Questão 10

Alterar o objeto x trocar a referência

Crie uma classe Caixa com o atributo privado `_valor`. Depois, implemente duas funções: `alterarValor(Caixa c)`, que modifica o valor interno do objeto recebido, e `trocarCaixa(Caixa c)`, que cria uma nova Caixa e a atribui à variável local `c`. No main, crie uma caixa, chame as duas funções e mostre os resultados na tela.

```
void main() {  
    Caixa caixa = Caixa(10);  
  
    print('Estado inicial:');  
    caixa.exibir();  
  
    alterarValor(caixa);  
    print('\nDepois de alterarValor:');  
    caixa.exibir();  
  
    trocarCaixa(caixa);  
    print('\nDepois de trocarCaixa:');  
    caixa.exibir();  
}
```

Após executar o programa, responda:

- a) Por que `alterarValor()` modifica o objeto original?
- b) Por que `trocarCaixa()` não substitui a caixa criada no main?
- c) O que esse resultado mostra sobre a diferença entre alterar um objeto e reatribuir uma variável de referência?

Questão 11

Cliente com cópia manual

Crie uma classe Endereco com os atributos privados `_rua`, `_numero` e `_cidade`. Depois, crie uma classe Cliente com os atributos privados `_nome` e `_endereco`. Implemente um construtor nomeado `Cliente.copia(Cliente outro)`, que copie os dados do cliente, mas crie também um novo objeto Endereco com os mesmos valores do endereço original. No main, crie um cliente, faça a cópia e altere o endereço de apenas um deles. Exiba os dois resultados.

```
void main() {  
    Cliente c1 = Cliente('Marina', Endereco('Rua A', 10, 'Curitiba'));  
    Cliente c2 = Cliente.copia(c1);  
  
    c2._endereco.alterarRua('Rua B');  
  
    print('Cliente original:');  
    c1.exibir();  
  
    print('\nCliente copiado:');  
    c2.exibir();  
}
```

Após executar o programa, responda:

- a) Por que a alteração no endereço de um cliente não modificou o endereço do outro?
- b) O que foi feito nesse exercício para evitar o compartilhamento da mesma referência de Endereco?
- c) Qual é a diferença entre copiar a

referência de um objeto e criar uma nova instância c/ os mesmos dados?

Questão 12

Agenda com contatos

Crie uma classe Contato com os atributos privados `_nome` e `_telefone`. Depois, crie uma classe Agenda com dois construtores nomeados:

- `Agenda.vazia()`
- `Agenda.comContatos(List<Contato> contatos)`.

Implemente os métodos `adicionarContato()`, `removerContatoPorNome()` e `listarContatos()`. No main, teste as duas formas de criação da agenda e exiba os contatos cadastrados.

```
class Agenda {
    List<Contato> _contatos;

    Agenda.vazia() : _contatos = [];

    Agenda.comContatos(this._contatos);

    void adicionarContato(Contato contato) {
        _contatos.add(contato);
    }

    void removerContatoPorNome(String nome) {
        _contatos.removeWhere((c) => c._nome == nome);
    }

    void listarContatos() {
        for (var contato in _contatos) {
            contato.exibir();
        }
    }
}

void main() {
    Agenda agenda1 = Agenda.vazia();
    agenda1.adicionarContato(Contato('Ana', '1111-1111'));

    Agenda agenda2 = Agenda.comContatos([
        Contato('Bruno', '2222-2222'),
        Contato('Clara', '3333-3333'),
    ]);

    print('Agenda 1:');
    agenda1.listarContatos();

    print('\nAgenda 2:');
    agenda2.listarContatos();
}
```

Após executar o programa, responda:

a) Qual é a diferença prática entre criar uma agenda vazia e criar uma agenda já com contatos?

b) Quando uma lista de objetos é passada para o construtor, os contatos são recriados ou a agenda passa a usar as referências recebidas?

Pedido com item compartilhado

Questão 13

Simulação conceitual de heap e stack

Crie uma classe Carteira com o atributo privado `_saldo` e uma classe Pessoa com os atributos privados `_nome` e `_carteira`. No main, crie uma pessoa com uma carteira, atribua essa mesma instância a uma segunda variável, altere o saldo da carteira usando uma das variáveis e de-

pois atribua null às referências, uma de cada vez. Exiba o estado do programa em cada etapa com impressões na tela.

```

class Carteira {
    double _saldo;

    Carteira(this._saldo);

    void adicionar(double valor) {
        _saldo += valor;
    }

    void exibir() {
        print('Saldo: $_saldo');
    }
}

class Pessoa {
    String _nome;
    Carteira _carteira;

    Pessoa(this._nome, this._carteira);

    void exibir() {
        print('Pessoa: $_nome');
        _carteira.exibir();
    }
}

void main() {
    Pessoa? p1 = Pessoa('Lucas', Carteira(100));
    Pessoa? p2 = p1;

    print('Estado inicial:');
    p1!.exibir();
    p2!.exibir();

    p2._carteira.adicionar(50);

    print('\nDepois de alterar a carteira por p2:');
    p1.exibir();
    p2.exibir();

    p1 = null;
    print('\nDepois de p1 = null:');
    print('p1: $p1');
    p2.exibir();

    p2 = null;
    print('\nDepois de p2 = null:');
    print('p2: $p2');
}

```

Após executar o programa, responda:

- Por que a alteração feita na carteira por p2 também apareceu quando o objeto foi acessado por p1?
- Em termos conceituais, as variáveis p1 e p2 armazenam o objeto inteiro ou uma referência para ele?
- Considerando o funcionamento de memória em linguagens orientadas a objetos, onde ficam as variáveis locais e onde ficam os objetos criados dinamicamente?
- Em que momento os objetos Pessoa e Carteira passam a ficar elegíveis para o garbage collector?

Questão 14

Banco com conta compartilhada entre gerentes

Crie uma classe Conta com os atributos privados _titular e _saldo. Depois, crie uma classe Gerente com os métodos depositar(Conta conta, double valor) e sacar(Conta conta, double valor). No main, crie uma única conta e dois gerentes diferentes. Faça operações com os dois gerentes sobre a mesma conta e exiba o saldo após cada operação.

```

void main() {
    Conta conta = Conta('Fernanda', 1000);
    Gerente g1 = Gerente('Marcos');
    Gerente g2 = Gerente('Patrícia');

    print('Estado inicial:');
    conta.exibir();

    g1.depositar(conta, 200);
    conta.exibir();

    g2.sacar(conta, 150);
    conta.exibir();
}

```

Após executar o programa, responda:

- Por que os dois gerentes conseguem alterar o mesmo saldo?
- Cada gerente possui uma conta própria ou ambos estão operando sobre a mesma instância?

c) O que esse exercício mostra sobre compartilhamento de referência entre objetos?

Questão 15

Funcionário com múltiplos construtores

Crie uma classe `Funcionario` com os atributos privados `_nome`, `_cargo` e `_salario`. Implemente os construtores nomeados `Funcionario.estagiario(String nome)`, `Funcionario.pleno(String nome, double salario)` e `Funcionario.gerente({required String nome, double bonus = 1000})`. Adicione os métodos `calcularPagamentoMensal()` e `exibirDados()`. No main, crie objetos com cada uma das formas de construção e exiba seus dados.

```
void main() {  
    Funcionario f1 = Funcionario.estagiario('João');  
    Funcionario f2 = Funcionario.pleno('Larissa', 3500);  
    Funcionario f3 = Funcionario.gerente(nome: 'Carlos', bonus: 1500);  
  
    f1.exibirDados();  
    f2.exibirDados();  
    f3.exibirDados();  
}
```

Após executar o programa, responda:

a) Qual é a vantagem de usar construtores nomeados nesse caso?

b) O que muda no estado inicial de cada objeto conforme o construtor utilizado?

c) Em vez de criar métodos separados para configurar o cargo depois, por que pode ser melhor definir isso já no construtor?